The Black Friday Crash

Engineering FlashKart for a 50x Traffic Day — Without Failing Again

Perspective : *Chief Technology Officer*

Prepared By : *Keshav*

Scale Target : *20,000 daily users → 1,000,000 users in a single day*

"The platform cannot fail again." — 3 months, fixed budget.

1. TECHNICAL RISKS

What Actually Breaks at 50x Peak Traffic

Last sale's five symptoms all trace back to three root causes — fixing the symptoms individually would just move the failure point.

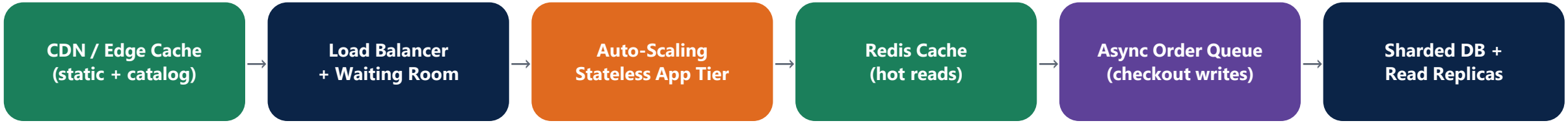
Symptom (Last Sale)	Root Cause	Why It Happens
Website crashes / slow loading	***Compute + database can't scale horizontally under a spike***	*Fixed server counts and one primary DB take the full weight of every request, including repetitive catalog reads.*
Failed payments	***Payment gateway calls made synchronously, no timeout/retry handling***	*A slow gateway response blocks the whole checkout thread instead of failing gracefully or retrying.*
Duplicate orders	***No idempotency on order/payment submission***	*A user's retried click (or a client-side retry after a timeout) is processed as a brand-new order.*
Customer complaints	***No real-time visibility or customer-facing status during failure***	*Users get a blank error with no explanation, and the team learns about the outage from complaints, not alerts.*

Three fixes — elastic compute/caching, async payment handling, and idempotent writes — address all five symptoms at the source.

2. INFRASTRUCTURE STRATEGY

Redesigning the Request Path for Scale

Design principle: 90%+ of peak traffic is browsing, not buying — cache it aggressively so only real purchase intent reaches the database.



Key Architecture Decisions

- ***Cache the catalog, not just the CDN:*** *product/browse pages served from CDN + Redis, so browsing traffic never touches the primary database.*
- ***Decouple checkout from fulfillment:*** *"place order" writes to a queue and returns immediately; inventory/fulfillment processes it asynchronously — smooths the spike instead of forcing the DB to absorb it in real time.*
- ***Auto-scale stateless compute only:*** *app servers hold no session state, so Kubernetes HPA (or equivalent) can add capacity in seconds without sticky-session complexity.*
- ***Shard or read-replica the database:*** *writes go to a right-sized primary; reads (the vast majority) are served from replicas.*

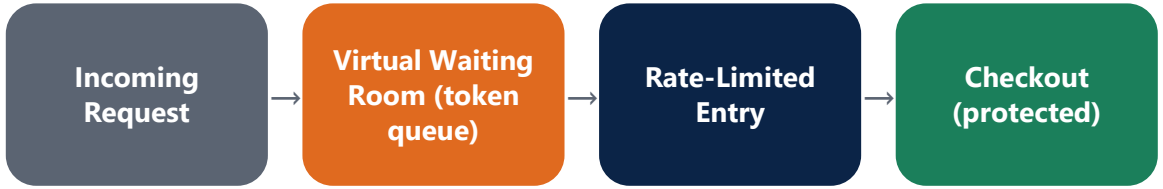
Trade-off, Stated Plainly

Async checkout means a customer's order confirmation may lag by a few seconds during peak load rather than being instant — an acceptable trade against the alternative of the entire site going down. This is disclosed on the confirmation screen ("processing your order") rather than hidden, which also softens the customer-complaints risk from Slide 2. With a 3-month timeline and fixed budget, this architecture reuses managed cloud services (managed Kubernetes, managed Redis/queue, managed Postgres read replicas) instead of building custom infrastructure — buy proven scale, don't build it from scratch under deadline.

3. RELIABILITY MEASURES

High Availability and Graceful Degradation

The goal isn't zero load-shedding — it's making sure that when something has to give, it's never checkout.



Users past capacity wait in a queue with a visible position/ETA instead of hammering a struggling backend — the same pattern large retailers use on drop days.

Other HA Measures

- ***Multi-AZ redundancy:*** *no single data-center failure takes the platform down.*
- ***Circuit breakers,*** *so one failing dependency (e.g. recommendations service) can't cascade into checkout.*

Graceful Degradation Ladder

First to go	*Recommendations, reviews, "related products" — non-critical, cache-safe to disable.*
Second	*Search relevance ranking falls back to simple/cached results.*
Never	*Checkout, payment, and order confirmation — protected above all else.*

Feature flags make each tier a single toggle, tested during load-test game days (Slide 7) so no one is deciding this live under pressure.

4. SECURITY & PAYMENTS

Protecting Customer Data and Money Under Load

A high-traffic sale is also the highest-value day of the year for an attacker — security and reliability are the same workstream here, not two.

Fixing Duplicate Orders & Failed Payments

- ***Idempotency keys*** *on every order/payment request — a retried click resolves to the same order, never a duplicate.*
- ***Multi-provider payment failover:*** *if the primary gateway times out, retry through a secondary provider instead of failing the sale outright.*

Protecting Customer Data

- ***Tokenized payments (PCI-DSS scope):*** *card data never touches FlashKart's own servers — handled by the payment processor's vault.*
- ***WAF + DDoS protection at the edge,*** *since a high-profile sale is a bigger target on the one day it can least afford downtime.*

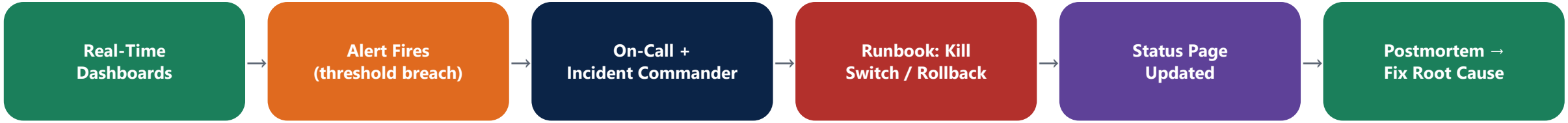
Protecting Inventory From Bots

Scalper bots checking out high-demand items faster than real customers is a security problem that shows up as a customer-trust problem — real users see "sold out" seconds after launch and blame the platform, not the bots. Rate limiting per account/IP and basic bot-detection heuristics on the checkout endpoint (paired with the waiting-room queue from Slide 4) keep the sale fair, which is as much a reliability measure for customer trust as it is a security control.

5. MONITORING & INCIDENT RESPONSE

If It Crashes Anyway, Here's What Happens

"It cannot fail again" also means: if it does degrade, the team's response is rehearsed, not improvised.



What's Being Watched Live

- ***Error rate & p95/p99 latency*** *per service — the earliest signal something's degrading.*
- ***Payment success rate*** *and queue depth on the order-processing pipeline.*
- ***Autoscaler activity,*** *so capacity additions are visible, not assumed.*

Roles, Defined Before Sale Day

Incident Commander (single decision-maker), subsystem leads for infra/payments/checkout, and a comms lead who owns the status page — assigned in advance, not volunteered live. Every degradation tier from Slide 4's ladder maps to a specific runbook step, so the team executes a checklist under pressure instead of debating options.

EXECUTION

The 3-Month Path to Sale Day

Month 1	→	Month 2	→	Month 3
• *Redesign request path (caching, queue, autoscaling)* • *Idempotency + payment failover implemented* • *Monitoring dashboards + alert thresholds live*		• *Load test at simulated 1M users* • *Chaos-engineering game days* • *rehearsing the incident runbook end to end*		• *Load test at 1.5x expected peak* • *Feature freeze on non-critical changes* • *Go/no-go review with roles + kill switches confirmed*

Budget Discipline

Every component above is a managed cloud service, not custom-built infrastructure — the fixed budget goes into engineering time on caching, queueing, and testing, not into standing up and operating new systems from scratch under a 3-month clock.

6. SUCCESS METRICS

How We'll Know It Actually Worked

Uptime during sale window

Target: 99.9%+ — effectively zero customer-visible downtime.

p95 page load time

Target: under 2 seconds, even at peak concurrent load.

Payment success rate

Target: 99%+, with automatic failover absorbing gateway blips.

Order duplication rate

Target: ~0% — the direct proof idempotency worked.

Autoscaling response time

Time from traffic spike to added capacity online.

Mean time to recovery (MTTR)

If an incident does occur, how fast the runbook resolves it.

Success isn't "nothing broke" — it's that every number above is visible in real time on sale day, so if something does start to degrade, it's caught and handled before a customer ever notices. That's what turns "the platform cannot fail again" from a mandate into an engineering plan.